

USE CLAUDE CODE

How Claude remembers your project

 Copy page

Give Claude persistent instructions with CLAUDE.md files, and let Claude accumulate learnings automatically with auto memory.

Each Claude Code session begins with a fresh context window. Two mechanisms carry knowledge across sessions:

- **CLAUDE.md files:** instructions you write to give Claude persistent context
- **Auto memory:** notes Claude writes itself based on your corrections and preferences

This page covers how to:

- Write and organize CLAUDE.md files
- Scope rules to specific file types with `.claude/rules/`
- Configure auto memory so Claude takes notes automatically
- Troubleshoot when instructions aren't being followed

CLAUDE.md vs auto memory

Claude Code has two complementary memory systems. Both are loaded at the start of every conversation. Claude treats them as context, not enforced configuration. To block an action regardless of what Claude decides, use a PreToolUse hook instead. The more specific and concise your instructions, the more consistently Claude follows them.

Scope	Project, user, or org	Per repository, shared across worktrees
Loaded into	Every session	Every session (first 200 lines or 25KB)
Use for	Coding standards, workflows, project architecture	Your preferences, corrections you give Claude, project context Claude can't derive from the code

Use CLAUDE.md files when you want to guide Claude's behavior. Auto memory lets Claude learn from your corrections without manual effort.

Subagents can also maintain their own auto memory. See [subagent configuration](#) for details.

CLAUDE.md files

CLAUDE.md files are markdown files that give Claude persistent instructions for a project, your personal workflow, or your entire organization. You write these files in plain text; Claude reads them at the start of every session.

When to add to CLAUDE.md

Treat CLAUDE.md as the place you write down what you'd otherwise re-explain. Add to it when:

- Claude makes the same mistake a second time
- A code review catches something Claude should have known about this codebase
- You type the same correction or clarification into chat that you typed last session
- A new teammate would need the same context to be productive

Keep it to facts Claude should hold in every session: build commands, conventions, project layout, "always do X" rules. If an entry is a multi-step procedure or only matters

CLAUDE.md files can live in several locations, each with a different scope. The table below lists them in load order, from broadest scope to most specific, so a project instruction appears in context after a user instruction.

Scope	Location	Purpose	Use case examples	Shared with
Managed policy	- macOS: <code>/Library/Application Support/ClaudeCode/CLAUDE.md</code> - Linux and WSL: <code>/etc/claude-code/CLAUDE.md</code> - Windows: <code>C:\Program Files\ClaudeCode\CLAUDE.md</code>	Organization-wide instructions managed by IT/DevOps	Company coding standards, security policies, compliance requirements	All users in organization
User instructions	<code>~/.claude/CLAUDE.md</code>	Personal preferences for all projects	Code styling preferences, personal tooling shortcuts	Just you (all projects)
Project instructions	<code>./CLAUDE.md</code> or <code>./.claude/CLAUDE.md</code>	Team-shared instructions for the project	Project architecture, coding standards, common workflows	Team members (source control)
Local instructions	<code>./CLAUDE.local.md</code>	Personal project-specific preferences; add to <code>.gitignore</code>	Your sandbox URLs, preferred test data	Just you (current project)

CLAUDE.md and CLAUDE.local.md files in the directory hierarchy above the working directory are loaded at launch. Files in subdirectories load on demand when Claude reads files in those directories. See [How CLAUDE.md files load](#) for the full resolution order.

A project CLAUDE.md can be stored in either `./CLAUDE.md` or `./.claude/CLAUDE.md`. Create this file and add instructions that apply to anyone working on the project: build and test commands, coding standards, architectural decisions, naming conventions, and common workflows. These instructions are shared with your team through version control, so focus on project-level standards rather than personal preferences. To confirm the file loaded, run `/context` in a session and check the list under **Memory files**.

💡 Run `/init` to generate a starting CLAUDE.md automatically. Claude analyzes your codebase and creates a file with build commands, test instructions, and project conventions it discovers. If a CLAUDE.md already exists, `/init` suggests improvements rather than overwriting it. Refine from there with instructions Claude wouldn't discover on its own.

Set `CLAUDE_CODE_NEW_INIT=1` to enable an interactive multi-phase flow. `/init` asks which artifacts to set up: CLAUDE.md files, skills, and hooks. It then explores your codebase with a subagent, fills in gaps via follow-up questions, and presents a reviewable proposal before writing any files.

Write effective instructions

CLAUDE.md files are loaded into the context window at the start of every session, consuming tokens alongside your conversation. The [context window visualization](#) shows where CLAUDE.md loads relative to the rest of the startup context. Because they're context rather than enforced configuration, how you write instructions affects how reliably Claude follows them. Specific, concise, well-structured instructions work best.

Size: target under 200 lines per CLAUDE.md file. Longer files consume more context and reduce adherence. If your instructions are growing large, use [path-scoped rules](#) so instructions load only when Claude works with matching files. You can also split content into [imports](#) for organization, though imported files still load and enter the context window at launch.

Specificity: write instructions that are concrete enough to verify. For example:

- “Use 2-space indentation” instead of “Format code properly”
- “Run `npm test` before committing” instead of “Test your changes”
- “API handlers live in `src/api/handlers/`” instead of “Keep files organized”

Consistency: if two rules contradict each other, Claude may pick one arbitrarily. Review your CLAUDE.md files, nested CLAUDE.md files in subdirectories, and `.claude/rules/` periodically to remove outdated or conflicting instructions. In monorepos, use `claudeMdExcludes` to skip CLAUDE.md files from other teams that aren't relevant to your work.

Import additional files

CLAUDE.md files can import additional files using `@path/to/import` syntax. Imported files are expanded and loaded into context at launch alongside the CLAUDE.md that references them.

Both relative and absolute paths are allowed. Relative paths resolve relative to the file containing the import, not the working directory. Imported files can recursively import other files, with a maximum depth of four hops.

Import parsing skips Markdown code spans and fenced code blocks. To mention a path in your CLAUDE.md without importing it, wrap it in backticks: writing ``@README`` keeps the text literal, while `@README` outside backticks imports the file.

To pull in a README, package.json, and a workflow guide, reference them with `@` syntax anywhere in your CLAUDE.md:

See `@README` for project overview and `@package.json` for available npm commands:

```
# Additional Instructions
```

```
- git workflow @docs/git-instructions.md
```

personal option does this for you.

If you work across multiple git worktrees of the same repository, a `gitignored` `CLAUDE.local.md` only exists in the worktree where you created it. To share personal instructions across worktrees, import a file from your home directory instead:

```
# Individual Preferences
- @~/ .claude/my-project-instructions.md
```

⚠ An import in a project-level memory file is external when its path resolves outside your working directory, like the home directory import above. The first time Claude Code encounters external imports in a project, it shows an approval dialog listing the files. If you decline, the imports stay disabled and the dialog doesn't appear again.

Claude Code shows the dialog to protect you from files other people commit to a shared project. User-scope memory files, such as `~/ .claude/CLAUDE.md` and `~/ .claude/rules/`, are files you wrote yourself. Except in **Cowork** sessions on your desktop, Claude Code loads their imports without the dialog and trusts them like the rest of your personal configuration.

In Cowork sessions on your desktop, Claude Code skips any import in a user-scope file that resolves to a path outside the session's working directory and loads the rest of the file. In those sessions it also skips a `~/ .claude/CLAUDE.md` that is itself a symlink or hard link, and a symlinked `~/ .claude/rules/` directory or rule file that points outside the working directory.

AGENTS.md

Claude Code reads `CLAUDE.md`, not `AGENTS.md`. If your repository already uses `AGENTS.md` for other coding agents, create a `CLAUDE.md` that imports it so both tools read the same instructions without duplicating them. You can also add Claude-specific instructions below the import. Claude loads the imported file at session start, then appends the rest:

Use plan mode for changes under ``src/billing/``.

A symlink also works if you don't need to add Claude-specific content:

```
ln -s AGENTS.md CLAUDE.md
```

The command prints no output on success. In your next session, run `/context` and confirm `CLAUDE.md` appears under **Memory files**.

On Windows, creating a symlink requires Administrator privileges or Developer Mode, so use the `@AGENTS.md` import instead.

Running `/init` reads Cursor rules, in `.cursor/rules/` or `.cursorrules`, and Copilot rules, in `.github/copilot-instructions.md`, and incorporates the relevant parts into the generated `CLAUDE.md`. With `CLAUDE_CODE_NEW_INIT=1` set, `/init` also reads `AGENTS.md`, `.devin/rules/`, `.windsurf/rules/` or `.windsurfrules`, and `.clinerules`.

You can also run `/import` to bring a supported coding agent's configuration into Claude Code, which appends a one-time copy of instruction files such as `AGENTS.md` to the matching `CLAUDE.md` and carries over MCP servers, commands, subagents, and skills. Requires Claude Code v2.1.213 or later.

How CLAUDE.md files load

Claude Code loads `CLAUDE.md` and `CLAUDE.local.md` from your current working directory and every directory above it. Run Claude Code in `foo/bar/` and it loads instructions from `foo/bar/CLAUDE.md`, `foo/CLAUDE.md`, and any `CLAUDE.local.md` files alongside them.

read last. Within each directory, `CLAUDE.local.md` is appended after `CLAUDE.md`, so your personal notes are the last thing Claude reads at that level.

Claude also discovers `CLAUDE.md` and `CLAUDE.local.md` files in subdirectories under your current working directory. Instead of loading them at launch, they are included when Claude reads files in those subdirectories.

If you work in a large monorepo where other teams' `CLAUDE.md` files get picked up, use `claudeMdExcludes` to skip them. For the full layout of root and per-directory `CLAUDE.md` files and rules, see [Monorepos and large repos](#).

Block-level HTML comments (`<!-- maintainer notes -->`) in `CLAUDE.md` files are stripped before the content is injected into Claude's context. Use them to leave notes for human maintainers without spending context tokens on them. Comments inside code blocks are preserved. When you open a `CLAUDE.md` file directly with the Read tool, comments remain visible.

Load from additional directories

The `--add-dir` flag gives Claude access to additional directories outside your main working directory. By default, `CLAUDE.md` files from these directories are not loaded.

To also load memory files from additional directories, set the

`CLAUDE_CODE_ADDITIONAL_DIRECTORIES_CLAUDE_MD` environment variable:

```
CLAUDE_CODE_ADDITIONAL_DIRECTORIES_CLAUDE_MD=1 claude --add-dir ../shared-co
```

This loads `CLAUDE.md`, `.claude/CLAUDE.md`, `.claude/rules/*.md`, and `CLAUDE.local.md` from the additional directory. `CLAUDE.local.md` is skipped if you exclude `local` from `--setting-sources`.

when Claude works with matching files, reducing noise and saving context space.

- ❗ Rules load into context every session or when matching files are opened. For task-specific instructions that don't need to be in context all the time, use [skills](#) instead, which only load when you invoke them or when Claude determines they're relevant to your prompt.

Set up rules

Place markdown files in your project's `.claude/rules/` directory. Each file should cover one topic, with a descriptive filename like `testing.md` or `api-design.md`. All `.md` files are discovered recursively, so you can organize rules into subdirectories like `frontend/` or `backend/`:

```
your-project/
├── .claude/
│   ├── CLAUDE.md           # Main project instructions
│   └── rules/
│       ├── code-style.md   # Code style guidelines
│       ├── testing.md      # Testing conventions
│       └── security.md     # Security requirements
```

Rules without `paths` **frontmatter** are loaded at launch with the same priority as `.claude/CLAUDE.md`.

Project rules are skipped if you exclude `project` from `--setting-sources`. Before v2.1.211, rules that load on demand, including path-scoped rules and rules in nested `.claude/rules/` directories, loaded even when `project` was excluded.

Path-specific rules

Rules can be scoped to specific files using YAML frontmatter with the `paths` field. These conditional rules only apply when Claude is working with files matching the

```
- "src/api/**/*.ts"
---
```

API Development Rules

- All API endpoints must include input validation
- Use the standard error response format
- Include OpenAPI documentation comments

Rules without a `paths` field are loaded unconditionally and apply to all files. Path-scoped rules trigger when Claude reads files matching the pattern, not on every tool use. As of v2.1.198, matching also works when Claude reaches a file through a symlinked path to the project directory, for example in a symlinked checkout.

Use glob patterns in the `paths` field to match files by extension, directory, or any combination:

Pattern	Matches
<code>**/*.ts</code>	All TypeScript files in any directory
<code>src/**/*.ts</code>	All files under <code>src/</code> directory
<code>*.md</code>	Markdown files in the project root
<code>src/components/*.tsx</code>	React components in a specific directory

You can specify multiple patterns and use brace expansion to match multiple extensions in one pattern:

```
- "tests/**/*.test.ts"  
---
```

Each brace group multiplies the number of expanded patterns: `src/*.{ts,tsx}` expands to two patterns, and `{a,b}/{c,d}/*.{ts,tsx}` to eight. To keep expansion bounded, a rule's whole `paths` list shares one budget of 1,000 expanded patterns and 4 MiB, and patterns without braces don't count against it.

Claude Code uses any pattern that would exceed the budget unexpanded, and its literal braces match no files. Before v2.1.217, a `paths` value with many brace groups stalled or crashed the CLI at startup.

Glob syntax treats `[` as the start of a bracket expression such as `[abc]`. A pattern with a `[` that can't be read as a bracket expression, such as `photos [2024/**`, is invalid: it matches nothing, and the rule's other patterns keep working. To match a literal `[` in a file name, escape it as `photos \[2024/**`. Before v2.1.207, one invalid pattern made the Read tool fail for every file the rule was evaluated against, instead of matching nothing.

Share rules across projects with symlinks

The `.claude/rules/` directory supports symlinks, so you can maintain a shared set of rules and link them into multiple projects. Symlinks are resolved and loaded normally, and circular symlinks are detected and handled gracefully.

This example links both a shared directory and an individual file:

```
ln -s ~/shared-claude-rules .claude/rules/shared  
ln -s ~/company-standards/security.md .claude/rules/security.md
```

```
~/.claude/rules/  
├─ preferences.md    # Your personal coding preferences  
└─ workflows.md     # Your preferred workflows
```

User-level rules are loaded before project rules, giving project rules higher priority.

Manage CLAUDE.md for large teams

For organizations deploying Claude Code across teams, you can centralize instructions and control which CLAUDE.md files are loaded.

Deploy organization-wide CLAUDE.md

Organizations can deploy a centrally managed CLAUDE.md that applies to all users on a machine. This file cannot be excluded by individual settings.

1 Create the file at the managed policy location

- macOS: `/Library/Application Support/ClaudeCode/CLAUDE.md`
- Linux and WSL: `/etc/claude-code/CLAUDE.md`
- Windows: `C:\Program Files\ClaudeCode\CLAUDE.md`

2 Deploy with your configuration management system

Use MDM, Group Policy, Ansible, or similar tools to distribute the file across developer machines. See [managed settings](#) for other organization-wide configuration options.

The `claudeMd` key lets you put managed CLAUDE.md content directly inside `managed-settings.json` instead of deploying a separate file.

CLAUDE.MD.

Where it's honored: managed and policy settings only. Setting `claudeMd` in user, project, or local settings has no effect.

The example below adds behavioral instructions directly in a managed settings file:

```
{
  "claudeMd": "Always run `make lint` before committing.\nNever push directly"
}
```

A managed CLAUDE.md and managed settings serve different purposes. Use settings for technical enforcement and CLAUDE.md for behavioral guidance:

Concern	Configure in
Block specific tools, commands, or file paths	Managed settings: <code>permissions.deny</code>
Enforce sandbox isolation	Managed settings: <code>sandbox.enabled</code>
Environment variables and API provider routing	Managed settings: <code>env</code>
Login method and organization restrictions	Managed settings: <code>forceLoginMethod</code> , <code>forceLoginOrgUUID</code>
Code style and quality guidelines	Managed CLAUDE.md
Data handling and compliance reminders	Managed CLAUDE.md
Behavioral instructions for Claude	Managed CLAUDE.md

Settings rules are enforced by the client regardless of what Claude decides to do. CLAUDE.md instructions shape Claude's behavior but are not a hard enforcement layer.

This example excludes a top-level CLAUDE.md and a rules directory from a parent folder. Add it to `.claude/settings.local.json` so the exclusion stays local to your machine:

```
{
  "claudeMdExcludes": [
    "**/monorepo/CLAUDE.md",
    "/home/user/monorepo/other-team/.claude/rules/**"
  ]
}
```

Patterns are matched against absolute file paths using glob syntax. You can configure `claudeMdExcludes` at any **settings layer**: user, project, local, or managed policy. Arrays merge across layers.

Managed policy CLAUDE.md files cannot be excluded. This ensures organization-wide instructions always apply regardless of individual settings.

Auto memory

Auto memory lets Claude accumulate knowledge across sessions without you writing anything. As it works, Claude saves four kinds of notes for itself. Claude records the kind as a `type` field in the memory file's frontmatter:

- `user` : your role, expertise, and working preferences
- `feedback` : corrections you give Claude and approaches you confirm
- `project` : ongoing work, deadlines, and decisions that Claude can't derive from the code or git history
- `reference` : where to find information outside the project, such as an issue tracker or dashboard

based on whether the information would be useful in a future conversation.

Enable or disable auto memory

Auto memory is on by default. To toggle it, open `/memory` in a session and use the auto memory toggle, which saves `autoMemoryEnabled` to your user settings at `~/.claude/settings.json`. To turn it off for a single project, set `autoMemoryEnabled` in that project's settings:

```
{
  "autoMemoryEnabled": false
}
```

To disable auto memory via environment variable, set

```
CLAUDE_CODE_DISABLE_AUTO_MEMORY=1
```

Storage location

Each project gets its own memory directory at

`~/.claude/projects/<project>/memory/`. The `<project>` path is derived from the git repository, so all worktrees and subdirectories within the same repo share one auto memory directory. Outside a git repo, the project root is used instead.

If you set `CLAUDE_CODE_PROJECT_DIR_NAME` beside `CLAUDE_CONFIG_DIR`, Claude Code uses that name as the `<project>` directory under `<config dir>/projects/` instead, whichever repository you launch it in, so projects launched with that config directory share one auto memory directory. Requires Claude Code v2.1.234 or later.

To store auto memory in a different location, set `autoMemoryDirectory` in your `settings.json`. It is read from any **settings scope**: user, project, local, policy, or `-- settings`.

The value must be an absolute path or start with `~/`. When you set it in a project's `.claude/settings.json` or `.claude/settings.local.json`, Claude Code honors it under the same **workspace trust rule as hooks in settings files**.

The directory contains a `MEMORY.md` index and one topic file per memory:

```
~/.claude/projects/<project>/memory/  
├─ MEMORY.md           # Index, one line per memory, loaded into every sess  
├─ user_role.md        # One memory  
├─ feedback_testing.md # One memory  
└─ ...                 # Any other topic files Claude creates
```

`MEMORY.md` acts as an index of the memory directory. Claude reads and writes files in this directory throughout your session, using `MEMORY.md` to keep track of what's stored where.

Auto memory is machine-local. All worktrees and subdirectories within the same git repository share one auto memory directory. Files are not shared across machines or cloud environments.

Claude Code deletes old session transcripts after the `cleanupPeriodDays` retention period, but excludes the files in the memory directory from that **retention sweep**.

`MEMORY.md` and topic files stay until you or Claude edits or deletes them.

How it works

The first 200 lines of `MEMORY.md`, or the first 25KB, whichever comes first, are loaded at the start of every conversation. Content beyond that threshold is not loaded at session start. Claude keeps `MEMORY.md` concise by moving detailed notes into separate topic files.

Claude to rewrite the index, because everything past the limit is dropped on the next load.

This limit applies only to `MEMORY.md`. Claude Code loads a `CLAUDE.md` file of up to 4 MiB in full and skips a larger file. Shorter files produce better adherence.

Claude Code doesn't load topic files such as `user_role.md` or `feedback_testing.md` at startup. Claude reads them on demand using its standard file tools when it needs the information.

The main conversation's auto memory isn't loaded into subagents; the exception is a fork, which inherits the parent conversation and system prompt. A subagent's own auto memory, enabled with the subagent `memory` field, is a separate directory.

Claude reads and writes memory files during your session. When you see messages like "Saved 2 memories" or "Recalled 2 memories" in the Claude Code interface, Claude is actively updating or reading from `~/.claude/projects/<project>/memory/`.

When Claude writes a memory file that begins with YAML frontmatter, Claude Code records the write time in a `modified` frontmatter field as an ISO 8601 timestamp. The timestamp shows how current the fact is, both to you and to Claude when it reads the memory back. Any file that has frontmatter gets the field the next time Claude writes it, including files created on earlier versions; Claude Code never adds frontmatter to a file that has none. The `modified` field requires Claude Code v2.1.214 or later.

Audit and edit your memory

Auto memory files are plain markdown you can edit or delete at any time. Run `/memory` to browse and open memory files from within a session.

entries for files that don't exist yet. It also lets you toggle auto memory on or off and provides an option to open the auto memory folder. Select any file to open it in your editor; selecting one that doesn't exist yet creates it first. To check which files actually loaded into the current session, run `/context`.

GUI editors such as VS Code open the file in a separate window, and you can keep using the session while it's open. Before v2.1.216, `/memory` waited for you to close the file before responding. Terminal editors such as Vim take over the terminal until you exit.

When you ask Claude to remember something, like “always use pnpm, not npm” or “remember that the API tests require a local Redis instance,” Claude saves it to auto memory. To add instructions to CLAUDE.md instead, ask Claude directly, like “add this to CLAUDE.md,” or edit the file yourself via `/memory`.

Troubleshoot memory issues

These are the most common issues with CLAUDE.md and auto memory, along with steps to debug them.

Claude isn't following my CLAUDE.md

CLAUDE.md content is delivered as a user message after the system prompt, not as part of the system prompt itself. Claude reads it and tries to follow it, but there's no guarantee of strict compliance, especially for vague or conflicting instructions.

To debug:

- Run `/context` and check the list under **Memory files** to verify your CLAUDE.md and CLAUDE.local.md files loaded. If a file is missing there, Claude can't see it. Use `/memory` to open and edit the files.
- Check that the relevant CLAUDE.md is in a location that gets loaded for your session (see [Choose where to put CLAUDE.md files](#)).

If the instruction is something that must run at a specific point, such as before every commit or after each file edit, write it as a **hook** instead. Hooks execute as shell commands at fixed lifecycle events and apply regardless of what Claude decides to do.

For instructions you want at the system prompt level, use `--append-system-prompt`. This must be passed every invocation, so it's better suited to scripts and automation than interactive use.

💡 Use the `InstructionsLoaded` **hook** to log exactly which instruction files are loaded, when they load, and why. This is useful for debugging path-specific rules or lazy-loaded files in subdirectories.

I don't know what auto memory saved

Run `/memory` and select the auto memory folder to browse what Claude has saved. Everything is plain markdown you can read, edit, or delete.

My CLAUDE.md is too large

Files over 200 lines consume more context and may reduce adherence. Claude Code skips a file over 4 MiB. Use **path-scoped rules** to load instructions only when Claude works with matching files, or trim content that isn't needed in every session. Splitting into `@path` **imports** helps organization but doesn't reduce context, since imported files load at launch.

The `/doctor` checkup proposes trims for a checked-in CLAUDE.md: it cuts content Claude can derive from the codebase, such as directory layouts, dependency lists, and architecture overviews, and keeps pitfalls, rationale, and conventions that differ from tool defaults. The trim check requires Claude Code v2.1.206 or later.

If an instruction disappeared after compaction, it was given only in conversation, lives in a nested CLAUDE.md that hasn't reloaded yet, or is a path-scoped rule that hasn't matched a file since. Add conversation-only instructions to CLAUDE.md to make them persist. See [What survives compaction](#) for the full breakdown.

See [Write effective instructions](#) for guidance on size, structure, and specificity.

Related resources

- [Debug your configuration](#): diagnose why CLAUDE.md or settings aren't taking effect
- [Skills](#): package repeatable workflows that load on demand
- [Settings](#): configure Claude Code behavior with settings files
- [Subagent memory](#): let subagents maintain their own auto memory

Was this page helpful?

 Yes

 No

Company

[Anthropic](#)

[Careers](#)

[Economic Futures](#)

[Research](#)

[News](#)

[Trust center](#)

[Transparency](#)

Learn

[Courses](#)

[MCP connectors](#)

[Customer stories](#)

[Engineering blog](#)

[Events](#)

[Powered by Claude](#)

[Service partners](#)

[Startups program](#)

Help and security

[Availability](#)

[Status](#)

[Support center](#)

Terms and policies

[Privacy choices](#)

[Privacy policy](#)

[Disclosure policy](#)

[Usage policy](#)

[Commercial terms](#)

[Consumer terms](#)